

PARCIAL 3

Fecha de entrega: 24 de mayo de 2026, 11:59 p.m.

Ospedale

Dado el proyecto Ospedale, que contiene una implementación del sistema de un hospital donde se pueden registrar pacientes, registrar doctores, solicitar, aceptar, completar y cancelar citas y hospitalizaciones, asignar tratamientos y visualizar la información en tablas, el equipo de trabajo debe realizar lo siguiente:

1. Realizar un **fork** del repositorio en Github, para tener su propia copia del repositorio sobre la cual realizar el parcial.
 - a. **Nota:** Es importante mencionar que luego de hacer el fork deben clonar el **nuevo repositorio** para trabajar en local. Una vez clonado el repositorio se debe ejecutar **inmediatamente** para realizar el cargue de las librerías correspondientes.
2. Hacer un refactor del proyecto para llevarlo a una arquitectura **MVC** (Model View Controller) utilizando los principios de **SOLID**.
 - a. Para las **vistas**:
 - i. **NO** modifique el aspecto visual del proyecto, i.e., no añada nuevos componentes gráficos ni mueva de lugar los ya existentes.
 - ii. Debe renombrar cada uno de los componentes gráficos para ofrecer una mayor claridad sobre la función que desempeña cada uno.
 - iii. **NO** se deben realizar verificaciones de los datos de entrada del lado de la vista.
 - iv. El usuario se debe mover de vista según corresponda:
 - Al realizar el ingreso con un usuario administrador se debe mover a la vista del administrador.
 - Al realizar el ingreso con un usuario paciente se debe mover a la vista del paciente.
 - Al realizar el ingreso con un usuario docto se debe mover a la vista de doctor.
 - Al hacer logout con cualquier tipo de usuario se debe mover a la vista de login.
 - Un usuario administrador debe acceder a las vistas de paciente y doctor enviando la información correspondiente.
 - Un usuario administrador debe volver a la vista del administrador desde las vistas de paciente y doctor. El botón back solo debe estar activo para un usuario administrador.
 - v. La información del usuario que ha ingresado a la plataforma se debe cargar automáticamente en los componentes visuales correspondientes.
 - vi. La información de los ComboBox se debe cargar automáticamente y en algunos casos depende de las opciones seleccionadas.
 - vii. Se debe invocar a los controladores y esperar una respuesta de estos.

- viii. Al obtener una respuesta, debe notificar al usuario del resultado de la operación (sea exitosa o no).
 - ix. Si la respuesta es exitosa, se debe limpiar la información de los componentes visuales correspondientes.
 - x. La vista **NO** debe ejecutarse a sí misma, i.e., la vista se debe ejecutar mediante un archivo **Main** distinto.
- b. Para los **controladores**:
- i. Utilizar el sistema de respuestas y códigos de estado del ejemplo desarrollado en clase.
 - ii. Realizar los controladores que permitan:
 - Realizar el login a la plataforma.
 - Registrar pacientes.
 - Registrar doctores.
 - Actualizar la información de un paciente.
 - Actualizar la información de un doctor.
 - Solicitar una cita u hospitalización.
 - Aceptar una cita u hospitalización.
 - Completar una cita.
 - Cancelar una cita u hospitalización.
 - Reprogramar una cita.
 - Prescribir medicamentos durante una cita.
 - Obtener la información necesaria para su correcta visualización en las tablas correspondientes.
 - iii. En los controladores a desarrollar se debe tener en cuenta los siguientes requerimientos:
 - Los **doctores** son registrados por el usuario **administrador**.
 - El **administrador** puede realizar cualquier operación como si fuera paciente y/o doctor.
 - Los **id** de los usuarios (pacientes y doctores) deben ser únicos, mayores que 0, tener 12 dígitos y **NO** pueden ser modificados.
 - Los **nombres de usuario** de los usuarios (administrador, pacientes y doctores) deben ser únicos y pueden ser modificados.
 - La **contraseña** y **confirmación de la contraseña** de los usuarios deben coincidir.
 - El **teléfono** de los pacientes debe tener exactamente 10 dígitos.
 - El **email** de los pacientes debe ser válido, i.e., debe seguir el formato **XXXXX@XXXXX.com**.
 - La **fecha de nacimiento** de los pacientes debe ser válida y seguir el formato **AAAA-MM-DD**.
 - El **número de licencia** de un doctor debe seguir el formato **L-XXXXXXXXXX MTL**, donde cada **X** es un dígito.
 - La oficina asignada a un doctor debe seguir el formato **O-XXX**, donde cada **X** es un dígito.
 - El **actualizar la información** de un paciente tiene los mismos requerimientos que la creación de un paciente.
 - El **actualizar la información** de un doctor tiene los mismos requerimientos que la creación de un doctor.
 - Asuma que todas las citas tienen una duración de **15 minutos**.
 - El id de una cita debe ser generado de manera automática y debe seguir el formato **A-{id_paciente}-NNNN**, donde

id_paciente es el id del paciente que solicita la cita y **NNNN** es un consecutivo de 4 dígitos que empieza en 0000 y debe ir incrementando progresivamente, e.g., para un paciente con id 112234567890, los id de sus citas serían: **A-112234567890-0000**, **A-112234567890-0001**, **A-112234567890-0002**, etc.

- La fecha de una cita debe ser válida y seguir el formato **AAAA-MM-DD**.
- La hora de la cita debe ser válida, seguir el formato de 24 horas **hh:mm** y los minutos deben ser en cuartos de hora, i.e., mm = {00, 15, 30, 45}.
- Al **realizar una operación que requiera de un doctor**, se debe garantizar que el doctor sea válido.
- Si un paciente solicita una cita con un doctor específico, el doctor en cuestión debe tener disponibilidad en el horario solicitado. En caso contrario, la cita no puede ser solicitada. Además, la especialidad de la cita debe ser la especialidad del doctor.
- Si un paciente solicita una cita por especialidad, se debe asignar a un doctor que tenga esa especialidad y que tenga disponibilidad en el horario solicitado. En caso contrario, la cita no puede ser solicitada.
- Todas las citas empiezan en estado REQUESTED.
- Al **realizar una acción sobre una cita** se debe garantizar que la cita sea válida.
- Un doctor puede aceptar una cita y su estado cambiará a PENDING.
- Un doctor puede completar una cita y su estado cambiará a COMPLETED.
- Un paciente puede cancelar una cita que no esté en estado COMPLETED y su estado cambiará a CANCELED.
- Un doctor puede reagendar una cita a una nueva hora que debe seguir el mismo formato de 24 horas **hh:mm** y las mismas reglas respecto a los minutos, no se puede cambiar el día de la cita. Además, puede ingresar una razón del reagendamiento de la cita, que debe ser añadida a la razón original de la solicitud de la esa cita.
- Un doctor puede prescribir una o varias medicaciones durante una cita. Sin embargo, solo podrá prescribir medicamentos si la cita ha sido aceptada (PENDING) y no ha sido completada (COMPLETED) o cancelada (CANCELED).
- El id de una hospitalización debe ser generado de manera automática y debe seguir el formato **H-{id_paciente}-NNNN**, donde **id_paciente** es el id del paciente que solicita la hospitalización y **NNNN** es un consecutivo de 4 dígitos que empieza en 0000 y debe ir incrementando progresivamente, e.g., para un paciente con id 112234567890, los id de sus hospitalizaciones serían: **H-112234567890-0000**, **H-112234567890-0001**, **H-112234567890-0002**, etc.
- La fecha de una hospitalización debe ser válida y seguir el formato **AAAA-MM-DD**.

- Todas las hospitalizaciones empiezan en estado REQUESTED si son solicitadas por un paciente.
 - Al **realizar una acción sobre una hospitalización** se debe garantizar que la hospitalización sea válida.
 - Un doctor puede aprobar una hospitalización y su estado cambiará a ONGOING.
 - Un doctor puede enviar a un paciente a una hospitalización directamente desde una cita y en ese caso la cita se da por completada (COMPLETED) y la hospitalización empieza en estado ONGOING.
 - Un doctor puede denegar una solicitud de hospitalización y su estado cambiará a CANCELED.
 - Las **citas** de un **paciente** se deben obtener de manera **ordenada descendentemente** (respecto a la **fecha y hora** de la cita), i.e., las citas más recientes deben ir primero.
 - Las **citas** de un **doctor** se deben obtener de manera **ordenada descendentemente** (respecto a la **fecha y hora** de la cita), i.e., las citas más recientes deben ir primero. Además, se debe tener en consideración lo solicitado desde la vista (citas totales o citas pendientes).
- iv. Al retornar una response, **NO** se debe enviar objetos de los modelos a la vista, en su lugar se debe enviar la información serializada del objeto.
- c. Para los **modelos**:
- i. Diseñe los modelos necesarios siguiendo los principios SOLID.
 - ii. En caso de ser necesario, simule un almacenamiento como en el ejemplo desarrollado en clase.
 - iii. Cargar la información de los archivos JSON que se encuentran en el repositorio y crear los objetos correspondientes. Además, se deben realizar las relaciones entre los objetos de manera adecuada en caso de ser necesario.
 - **Nota:** Utilizar la librería org.json que ya se encuentra cargada en el repositorio.
3. **(Bonificación)** Implemente los principios SOLID en los controladores del proyecto.
4. **(Bonificación)** Implemente adecuadamente el patrón observador para que cada vez que haya una creación o modificación sobre alguno de los modelos la tabla correspondiente se actualice automáticamente.

Criterios de calificación

Rubrica MVC y SOLID (3.0)

	0	1	2	3	4	5
MVC (2,0)	No implementa el patrón arquitectural MVC.	Implementa parcialmente un componente del patrón MVC.	Implementa dos componentes del patrón MVC, con separación limitada de responsabilidades.	Implementa los tres componentes del patrón MVC, con separación básica de responsabilidades.	Implementa correctamente los tres componentes del patrón MVC, con separación adecuada de responsabilidades.	Realiza una correcta separación de responsabilidades utilizando el patrón arquitectural MVC y cumple con todos los requerimientos solicitados para cada componente.
SOLID (1,0)	No cumple con ningún principio de SOLID.	Cumple con un principio de SOLID.	Cumple con dos principios de SOLID.	Cumple con tres principios de SOLID.	Cumple con cuatro principios de SOLID.	Cumple con todos los principios de SOLID.

Sustentación del parcial (2.0)

El viernes **29 de mayo de 2026** se realizará la sustentación del parcial en los siguientes horarios:

- 8:00 a.m. a 12:00 p.m.
- 2:00 p.m. a 5:00 p.m.

Posteriormente, se les estará comunicando el horario específico y el lugar en el que se deben presentar para la sustentación.

Nota: Todos los integrantes del grupo deben estar presentes el día de la sustentación.

Bonificación (1.0)

- Implementa adecuadamente los principios SOLID en los controladores del proyecto (0.5)
- Implementa adecuadamente el patrón observador para la actualización automática de las tablas (0.5)

Método e indicaciones para la entrega

- Este parcial es para desarrollar en **parejas** o en **grupos de 3 integrantes**, todo tipo de fraude será castigado según el reglamento.
 - Los grupos pueden estar conformados por personas de distintos NRC.
 - El registro de los integrantes de los grupos en el Brightspace se realizará durante las clases de la **semana 16**. Recuerde que para poder realizar el envío debe estar registrado en uno de estos equipos.
- Al momento de calificar se revisará el **último commit** realizado antes de la hora límite.
- Se deberá realizar un **fork** del repositorio de **Github** donde deberán cargar el código desarrollado.
 - Se debe enviar el enlace del repositorio por la actividad correspondiente en el curso de **Brightspace** a más tardar antes de la hora límite del parcial.
 - Los **nombres completos** de los integrantes y el **NRC** al que pertenecen deberán estar escritos en el archivo **README.md**.
 - **Todos** los integrantes deben tener **commits** en el repositorio, en caso contrario se asumirá que aquel que tenga todos los commits fue el único que trabajó.
- El incumplimiento de alguna de estas indicaciones implicará una penalización a la nota final del examen.